SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Integrated Experiments

Weightage: 40%

QUESTIONS: **Total Marks: 30**

Experiment 1: Responsive Layout Design

Suppose that a retail website is not responsive across mobile and tablet devices, causing misalignment of content and improper spacing.
(a) Analyze the possible reasons for lack of responsiveness in the existing layout.
(b) Design a responsive webpage layout using **CSS Flexbox or Grid**, ensuring proper alignment of header, navigation, content, and footer.
(c) Demonstrate how the layout adapts to **different screen sizes (mobile, tablet, desktop)** using media queries.
(d) Implement spacing, alignment, and distribution techniques to improve visual consistency.
(e) Evaluate the effectiveness of the responsive design and suggest further improvements.

Experiment 2: Form Validation using JavaScript

Suppose that a registration form accepts invalid email and phone number inputs, leading to incorrect data submission.
(a) Analyze the issues in the existing form validation mechanism.
(b) Design input fields for email and phone number with appropriate HTML attributes.
(c) Implement **JavaScript validation using Regular Expressions** for both email and phone number.

(d) Develop a mechanism to display **real-time error messages** for invalid inputs.
(e) Evaluate the validation system and suggest enhancements for better user experience and security.

Experiment 3: CSS Layout Debugging

Suppose that a webpage layout overlaps when the browser window is resized, affecting usability.
(a) Analyze the CSS properties responsible for layout overlapping issues.
(b) Examine the role of the **CSS box model (margin, border, padding, content)** in layout design.
(c) Debug the layout using appropriate **positioning techniques (relative, absolute, flex, grid)**.
(d) Modify the CSS to ensure proper alignment and prevent overlapping during resizing.
(e) Evaluate the corrected layout and suggest best practices for maintaining consistency.

Code of Conduct:

I ***Narjala Saptha Nageswar - 192372001*** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: *N Saptha Nageswar*

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools

Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

ANSWERS

Internet Programming (CSA43) – Assessment Tool 1: Integrated Experiments

Narjala Saptha Nageswar – 192372001

Experiment 1: Responsive Layout Design

(a) Analysis: reasons for lack of responsiveness

- **Missing viewport meta tag.** Without `<meta name="viewport" content="width=device-width, initial-scale=1.0">` mobile browsers render the page at about 980px and shrink it, so text becomes tiny and spacing looks wrong.
- **Fixed widths in px** (e.g. `width: 960px`) cause horizontal scrolling and cut-off content on screens narrower than the layout.
- **Absolute positioning and floats** for the main layout take elements out of the normal flow, so they overlap or collapse when the width changes.
- **No media queries**, so the same desktop layout is applied to every device.
- **Inconsistent box model:** without `box-sizing: border-box`, padding and border are added on top of the width and push elements past their containers.
- **Non-fluid images and fixed spacing** (images without `max-width:100%`, hard-coded margins) break proportions on small screens.

(b) Design: responsive layout with CSS Grid and Flexbox

The page uses a **mobile-first** approach. **CSS Grid** with named `grid-template-areas` arranges the four page regions (header, navigation, content, footer). **Flexbox** lays out the navigation links, and a Grid with `repeat()` arranges the product cards. All sizes use fractions, percentages and CSS variables instead of fixed pixel widths.

Layout diagram (areas as defined in the CSS):

MOBILE (<600px)	TABLET (600-991px)	DESKTOP (≥992px)
Header Navigation (stacked) Content (1 column) Footer	Header Navigation (row) Content (2 columns) Footer	Header (full width) Sidebar nav | Content (4 columns) Footer (full width)

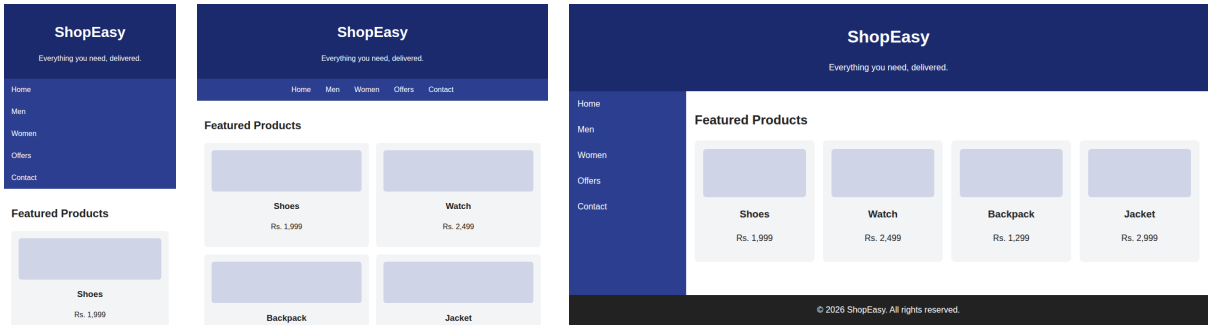
Complete code (exp1.html):

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Responsive Retail Layout</title>
<style>
  *, *::before, *::after { box-sizing: border-box; }
  :root { --gap: 16px; --brand: #1a2a6c; --light: #f3f4f6; }
  body { margin: 0; font-family: Arial, sans-serif; line-height: 1.5; color: #222;
         min-height: 100vh; display: grid;
         grid-template-areas: "header" "nav" "main" "footer";
         grid-template-rows: auto auto 1fr auto; }
  header { grid-area: header; background: var(--brand); color: #fff; padding: var(--gap); text-align: center; }
  nav    { grid-area: nav; background: #2c3e8f; }
  nav ul { list-style: none; margin: 0; padding: 0; display: flex; flex-direction: column; }
  nav a  { display: block; padding: 12px var(--gap); color: #fff; text-decoration: none; }
  nav a:hover { background: #3d52b3; }
  main   { grid-area: main; padding: var(--gap); }
  .products { display: grid; gap: var(--gap); grid-template-columns: 1fr; }
  .card { background: var(--light); border-radius: 8px; padding: var(--gap); text-align: center; }
  .card .img { height: 90px; background: #cfd4e6; border-radius: 6px; margin-bottom: 8px; }
  footer { grid-area: footer; background: #222; color: #fff; padding: var(--gap); text-align: center; }

  /* Tablet: 600px and above */
  @media (min-width: 600px) {
    nav ul { flex-direction: row; justify-content: center; flex-wrap: wrap; }
    .products { grid-template-columns: repeat(2, 1fr); }
  }
  /* Desktop: 992px and above */
  @media (min-width: 992px) {
    body { grid-template-areas: "header header" "nav main" "footer footer";
           grid-template-columns: 220px 1fr; grid-template-rows: auto 1fr auto; }
    nav ul { flex-direction: column; }
    .products { grid-template-columns: repeat(4, 1fr); }
  }
</style>
</head>
<body>
  <header><h1>ShopEasy</h1><p>Everything you need, delivered.</p></header>
  <nav><ul>
    <li><a href="#">Home</a></li><li><a href="#">Men</a></li>
    <li><a href="#">Women</a></li><li><a href="#">Offers</a></li><li><a href="#">Contact</a></li>
  </ul></nav>
  <main>
    <h2>Featured Products</h2>
    <section class="products">
      <article class="card"><div class="img"></div><h3>Shoes</h3><p>Rs. 1,999</p></article>
      <article class="card"><div class="img"></div><h3>Watch</h3><p>Rs. 2,499</p></article>
      <article class="card"><div class="img"></div><h3>Backpack</h3><p>Rs. 1,299</p></article>
      <article class="card"><div class="img"></div><h3>Jacket</h3><p>Rs. 2,999</p></article>
    </section>
  </main>
  <footer>&copy; 2026 ShopEasy. All rights reserved.</footer>
</body>
</html>
```

(c) Adaptation to mobile, tablet and desktop using media queries

Device	Breakpoint	Navigation	Product cards	Page structure
Mobile	default (< 600px)	Vertical list (flex-direction: column)	1 column	Single column; header, nav, main, footer stacked
Tablet	`@media (min-width: 600px)`	Horizontal row, centred, wraps if needed	2 columns	Still stacked, but content uses the extra width
Desktop	`@media (min-width: 992px)`	Vertical sidebar (220px)	4 columns	Two-column grid: sidebar + content; header and footer span both columns



Output at 375px (mobile), 768px (tablet) and 1200px (desktop)

(d) Spacing, alignment and distribution techniques used

- **CSS variable** `--gap` is used for all padding and grid gaps, so spacing stays consistent everywhere.
- `box-sizing: border-box` applied to all elements so padding/border never push elements wider than their container.
- **Grid** `gap` distributes the product cards evenly, avoiding margin hacks.
- **Flexbox** `justify-content: center` aligns the tablet navigation, and `text-align: center` aligns the header, cards and footer.
- `min-height: 100vh` **with** `grid-template-rows: auto auto 1fr auto` keeps the footer at the bottom even when the content is short.

(e) Evaluation and further improvements

Criterion	Result
No horizontal scroll at 375px / 768px / 1200px	Passed – fluid units and grid used
Header, nav, content and footer aligned at every size	Passed – controlled through grid-template-areas
Touch-friendly navigation on mobile	Passed – links are full-width blocks with 12px padding
Consistent spacing	Passed – single --gap variable

Suggested improvements: use real product images with `max-width:100%; height:auto` and `srcset` / lazy loading; add a hamburger menu for mobile; use `clamp()` for fluid font sizes; use `repeat(auto-fit, minmax(220px, 1fr))` to reduce the number of breakpoints; add `:focus-visible` styles and ARIA labels for accessibility; test on real devices and with Lighthouse.

Experiment 2: Form Validation using JavaScript

(a) Analysis of issues in the existing form validation

- The form relies on plain text inputs with **no validation attributes** (no `type="email"`, `required`, `pattern`), so any text is accepted.
- **No JavaScript pattern check**, so values like `abc@` or `12345` are submitted.
- **No feedback to the user**: errors, if any, appear only after submission, or not at all.
- The phone field accepts letters, spaces and the wrong length, and the mobile keyboard is not optimised for numbers.
- **Client-only checks can be bypassed**, and without server-side validation invalid or malicious data can reach the database.

(b) Design of input fields with appropriate HTML attributes

Attribute	Email field	Phone field
type	`email` (email keyboard, built-in check)	`tel` (numeric keypad on mobile)
required	Yes	Yes
pattern / maxlength	`maxlength="100"`	`pattern="[6-9][0-9]{9}"`, `maxlength="10"`
Other	`autocomplete="email"`, placeholder, label linked by `for`	`inputmode="numeric"`, `autocomplete="tel"`, placeholder, label

(c) JavaScript validation using regular expressions

Field	Regular expression	Meaning
Email	`/^[A-Za-z0-9._%+-]+@[A-Za-z0-9-]+(\.[A-Za-z0-9-]+)*\.[A-Za-z]{2,}$/`	Local part (letters, digits, . _ % + -), then @, a domain name, and a top-level domain of 2 or more letters.
Phone	`/^[6-9][0-9]{9}$/`	Exactly 10 digits, starting with 6, 7, 8 or 9 (Indian mobile number).

(d) Real-time error messages

An `input` event listener runs the validation on every keystroke. The field border turns green or red, an error message appears under the field, non-digit characters are removed from the phone field, and the Register button is enabled only when both fields are valid.

Complete code (exp2.html):

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Registration Form Validation</title>
<style>
  * { box-sizing: border-box; }
  body { font-family: Arial, sans-serif; background: #f3f4f6; display: flex; justify-content: center; padding: 30p
  form { background: #fff; padding: 24px; width: 100%; max-width: 380px; border-radius: 8px; box-shadow: 0 2px 8px
  label { display: block; margin-top: 14px; font-weight: bold; }
  input { width: 100%; padding: 10px; margin-top: 6px; border: 2px solid #ccc; border-radius: 6px; font-size: 15px
  input.valid   { border-color: #2e8b57; }
  input.invalid { border-color: #d32f2f; }
  .error { color: #d32f2f; font-size: 13px; min-height: 18px; margin-top: 4px; }
  button { margin-top: 18px; width: 100%; padding: 11px; border: 0; border-radius: 6px;
           background: #1a2a6c; color: #fff; font-size: 16px; cursor: pointer; }
  button:disabled { background: #9aa3c7; cursor: not-allowed; }
</style>
</head>
<body>
<form id="regForm" novalidate>
  <h2>Register</h2>
  <label for="email">Email</label>
  <input type="email" id="email" name="email" placeholder="name@example.com"
         required maxlength="100" autocomplete="email">
  <div class="error" id="emailError"></div>

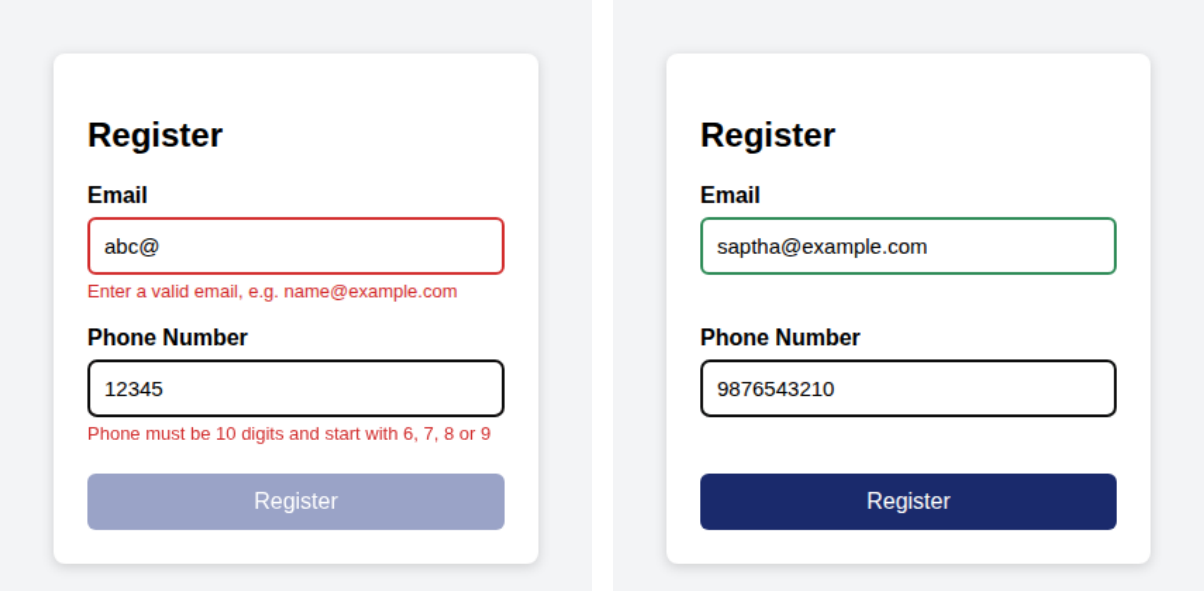
  <label for="phone">Phone Number</label>
  <input type="tel" id="phone" name="phone" placeholder="9876543210"
         required maxlength="10" inputmode="numeric" pattern="[6-9][0-9]{9}" autocomplete="tel">
  <div class="error" id="phoneError"></div>

  <button type="submit" id="submitBtn" disabled>Register</button>
</form>

<script>
  const emailRegex = /^[A-Za-z0-9._%+-]+@[A-Za-z0-9-]+(\.[A-Za-z0-9-]+)*\.[A-Za-z]{2,}$/;
  const phoneRegex = /^[6-9][0-9]{9}$/;          // 10-digit Indian mobile number

  const form = document.getElementById("regForm");
  const email = document.getElementById("email");
  const phone = document.getElementById("phone");
  const btn = document.getElementById("submitBtn");

  function check(input, regex, errorId, message) {
    const value = input.value.trim();
    const box = document.getElementById(errorId);
    if (value === "") {
      box.textContent = ""; input.className = ""; return false;
    }
    if (regex.test(value)) {
      box.textContent = ""; input.className = "valid"; return true;
    }
    box.textContent = message; input.className = "invalid"; return false;
  }
  function validateAll() {
    const e = check(email, emailRegex, "emailError", "Enter a valid email, e.g. name@example.com");
    const p = check(phone, phoneRegex, "phoneError", "Phone must be 10 digits and start with 6, 7, 8 or 9");
    btn.disabled = !(e && p);
    return e && p;
  }
  email.addEventListener("input", validateAll);       // real-time feedback
  phone.addEventListener("input", () => {
    phone.value = phone.value.replace(/\D/g, "");     // allow digits only
    validateAll();
  });
  form.addEventListener("submit", (ev) => {
    ev.preventDefault();
    if (validateAll()) alert("Registration successful!");
  });
</script>
</body>
</html>
```



Output: invalid input shows red error messages and a disabled button (left); valid input enables the button (right)

Test input	Field	Result
abc@	Email	Invalid – error shown
saptha@example.com	Email	Valid – green border
12345	Phone	Invalid – must be 10 digits
1234567890	Phone	Invalid – must start with 6-9
9876543210	Phone	Valid – green border
98abc	Phone	Letters removed automatically

(e) Evaluation and enhancements

Evaluation: the validation correctly blocks malformed email addresses and phone numbers, gives instant feedback, and prevents submission of invalid data. Limitation: the email regex checks only the format, not whether the mailbox exists, and client-side checks alone can be bypassed.

Enhancements for better user experience:

- Show a success tick, a clear message such as 'Phone must be 10 digits', and associate errors with inputs using `aria-describedby` and `aria-live` for screen readers.
- Validate on `blur` as well, so empty required fields are reported when the user leaves them.
- Support country codes using a library such as libphonenumber, and offer autofill.

Enhancements for security:

- **Always repeat the validation on the server**, and use parameterised queries to prevent SQL injection.
- Sanitise and escape the data before display (prevents XSS), and send the form over HTTPS.
- Verify the email using a confirmation link or OTP, and add CAPTCHA and rate limiting against bots.

Experiment 3: CSS Layout Debugging

(a) CSS properties responsible for the overlapping

Property / practice	Why it causes overlap
`position: absolute` with fixed `top / left` values	Elements are removed from the normal flow. The browser no longer reserves space for them, so they ignore each other and the sidebar (width 250px) and the content (starting at left: 220px) overlap.
Fixed pixel `width / height`	The 600px content box does not shrink when the window is resized and runs off the screen; fixed heights cause text to overflow.
Default `content-box` sizing	Padding and border are added to the width, so a 250px sidebar actually occupies 300px (250 + 2x20 + 2x5), making the overlap larger than planned.
Wrong `top` values and missing `z-index` planning	The sidebar (top: 40px) overlaps the 60px header, and the footer is placed at a fixed top value that does not move with the content.
No media queries	The layout does not change on narrow screens.

(b) Role of the CSS box model

Every element is a rectangular box made of four layers, from inside to outside: **content** (text or images), **padding** (space between content and border, inside the background), **border** (the edge of the box) and **margin** (transparent space separating the box from its neighbours).

Layer	Role in layout design	Effect on total size (content-box)
Content	Holds the actual content; set by width / height	width
Padding	Breathing space inside the element; shows the background colour	+ left and right padding
Border	Visible outline; separates the element	+ left and right border
Margin	Space between elements; margins of vertical siblings can collapse	Does not add to the element's own width, but adds to the space it occupies

Example: sidebar width 250px + padding 20px x 2 + border 5px x 2 = **300px** total with `content-box`, but exactly **250px** with `box-sizing: border-box`. Using border-box makes layouts predictable.

(c) Debugging using positioning techniques

Debugging steps: (1) open Browser DevTools (F12) and inspect each box to see its box-model diagram; (2) resize the window and watch which element overlaps; (3) find the properties that take elements out of the flow (absolute, fixed widths); (4) replace them with flow-based layout.

Technique	Behaviour	Recommended use
`relative`	Keeps its place in the flow; can be offset; becomes the anchor for absolute children	Containers that hold absolutely positioned badges or icons
`absolute`	Removed from the flow; positioned relative to the nearest positioned ancestor	Small overlays only (not main page layout)
`flex`	One-dimensional layout; items share space and never overlap	Header / sidebar + content rows, navigation bars
`grid`	Two-dimensional layout using rows and columns	Whole page structure and card galleries

Original (buggy) code – exp3_before.html:

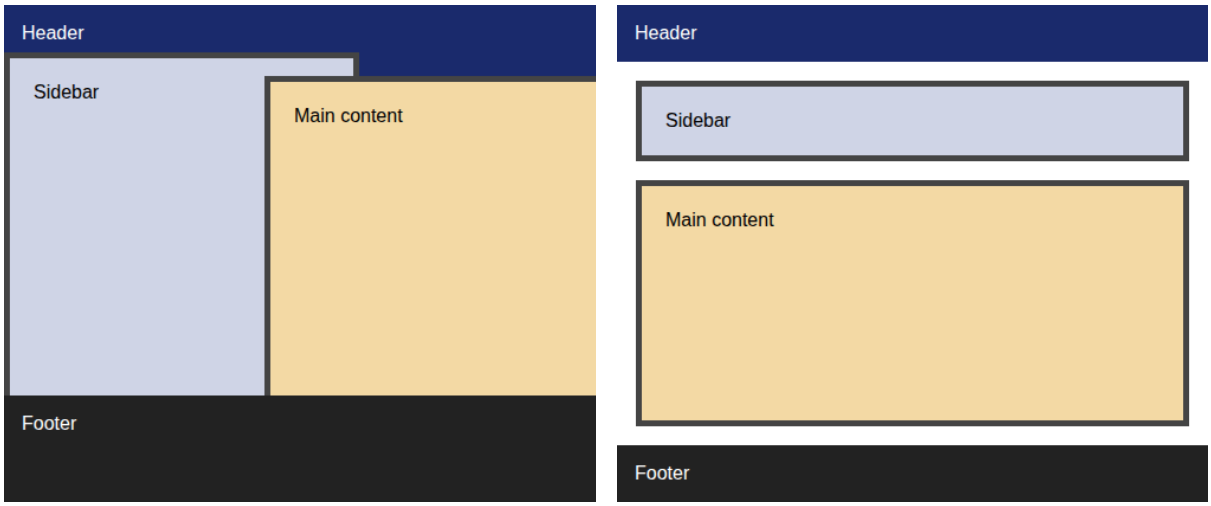
```
<!DOCTYPE html>
<html lang="en"><head><meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<style>
  body { margin: 0; font-family: Arial, sans-serif; }
  .header  { position: absolute; top: 0; left: 0; width: 100%; height: 60px; background: #1a2a6c; color:#fff; padd
  .sidebar { position: absolute; top: 40px; left: 0; width: 250px; height: 300px; background: #cfd4e6; padding: 20
  .content { position: absolute; top: 60px; left: 220px; width: 600px; height: 300px; background: #f3d9a4; padding
  .footer  { position: absolute; top: 330px; left: 0; width: 100%; height: 60px; background: #222; color:#fff; pad
</style></head><body>
  <div class="header">Header</div>
  <div class="sidebar">Sidebar</div>
  <div class="content">Main content</div>
  <div class="footer">Footer</div>
</body></html>
```

(d) Modified CSS that prevents overlapping

The absolute positioning was replaced by a **flexbox** page (column direction) containing a **flex row** for the sidebar and the content. `box-sizing: border-box` was added, fixed widths became `flex` values, and a media query stacks the columns on narrow screens. `min-width: 0` prevents the content from overflowing its flex container.

Corrected code – exp3_after.html:

```
<!DOCTYPE html>
<html lang="en"><head><meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<style>
  *, *::before, *::after { box-sizing: border-box; }   /* width includes padding + border */
  body { margin: 0; font-family: Arial, sans-serif; min-height: 100vh;
         display: flex; flex-direction: column; }
  .header  { background: #1a2a6c; color:#fff; padding: 15px; }
  .wrapper { flex: 1; display: flex; gap: 16px; padding: 16px; }
  .sidebar { flex: 0 0 250px; background: #cfd4e6; padding: 20px; border: 5px solid #444; }
  .content { flex: 1; min-width: 0; background: #f3d9a4; padding: 20px; border: 5px solid #444; }
  .footer  { background: #222; color:#fff; padding: 15px; }
  @media (max-width: 700px) {
    .wrapper { flex-direction: column; }
    .sidebar { flex-basis: auto; }
  }
</style></head><body>
  <div class="header">Header</div>
  <div class="wrapper">
    <div class="sidebar">Sidebar</div>
    <div class="content">Main content</div>
  </div>
  <div class="footer">Footer</div>
</body></html>
```



Left: before (sidebar and content overlap the header and each other). Right: after (elements stack cleanly at 500px)

(e) Evaluation and best practices

Test	Before	After
Overlap between header, sidebar and content	Yes	No
Horizontal scrolling at 500px width	Yes (content 600px wide)	No
Footer position	Fixed, overlaps content	Always after content
Sidebar actual width	300px (content-box)	250px (border-box)
Adapts on resize	No	Yes (flex + media query)

Best practices for maintaining consistency:

- Apply `*, *::before, *::after { box-sizing: border-box; }` at the start of every stylesheet, and use a CSS reset or normalise.
- Use **Flexbox / Grid** for page layout and keep absolute positioning for small overlays only.
- Prefer relative units (%, fr, rem, vw) and `max-width` to fixed pixel sizes, and avoid fixed heights.
- Use CSS variables for spacing and colours, and a mobile-first approach with media queries.
- Use meaningful class names (such as BEM) and avoid deep selector nesting and `!important`.
- Regularly test with DevTools and different screen widths and browsers, and validate the CSS.